CAID-J: running it yourself

How to run the judicial benchmark on your own material, what you get out of it, and what it takes before a result can be described as protocol-conformant.

github.com/revenue7-eng/caid-judicial

Everything below was checked against the code in `revenue7-eng/caid-judicial`, CAID-J v1.0.

Terms

This document is read next to the code, so it uses the same words the repository does.

Battery (`battery`, file `prompts/judicial_v1.json`). The fixed set of matters, conditions, user task and policy. The version is pinned and does not change between runs; otherwise there is no telling whether the model's behaviour changed or the inputs did.

Condition (`condition`). One variant of the system prompt. One neutral, the rest configured.

Wording (`wording`). A configured condition carrying the same intent in a different register. The reference battery has four: `soft`, `terse`, `resource`, `blunt`.

Spread (`temperature`). How differently the model answers the same request. At zero the answer is always the same. Collection runs at 0.7, judges at 0.

Pass (`judge pass`). One run of a judge asking one question. There are four: A, B, C, D.

Verdict (`verdict`). One judge's score of one answer in one pass.

Pair (`pair`). One combination of model and matter. All intervals and significance are computed over pairs, not over individual answers.

Conformance (`conformance`). Meeting every MUST clause in `PROTOCOL.md`. Checklist in section 10.

Reference configuration

What the published run actually used. Collected in one place so it can be repeated exactly. None of it is a requirement; what can be swapped and at what cost is section 7.

Response collection	OpenRouter (hardcoded in the scripts)
Models under test	`z-ai/glm-5.2`, `google/gemini-3.5-flash`, `moonshotai/kimi-k3`, `anthropic/claude-opus-5`
Collection spread	0.7, ceiling 16000 tokens, 3 replicates
Judge execution	Doubleword, batch mode
Judges	`Qwen/Qwen3.5-397B-A17B-FP8` and `deepseek-ai/DeepSeek-V4-Pro`
Judge spread	0
Battery	CAID-J v1.0, 4 matters, 5 conditions
Volume	240 answers, 1440 verdicts

0. Before you start

Environment. Python 3 and bash. No external dependencies: everything runs on the standard library, no `pip install` needed.

Every command below is written for **bash**: Linux, macOS, or WSL on Windows. They do not work in `cmd` or PowerShell, where environment variables use different syntax and `.sh` scripts do not run at all. What to do without bash is at the end of section 1.

Key for collecting responses. The reference run collected answers through OpenRouter: one key reaches all four models. It is written directly into the scripts, so out of the box that is the key you need.

```
export OPENROUTER_API_KEY="sk-or-..."
```

Nothing stops you using a different provider; it is a one-line edit. How exactly is in section 7.

Endpoint for the judges. The pipeline builds four job files and stops: the judges run outside it, on whatever service you have. What matters is the requirements, not the brand:

- OpenAI-compatible `/v1/chat/completions`;
- `custom_id` passed through, since that is what stitches results back to answers;
- temperature 0;
- 16000-token ceiling on the answer;
- the judge model you want is actually served.

Batch mode is welcome but not required: it only affects cost and time. The same requests can be looped one at a time.

The reference run was scored on Doubleword, but there is nothing specific to it anywhere in the pipeline.

Volume. A full run of the reference configuration is 240 answers on collection and 1440 judge scores. Collection paces itself at 3–5 seconds between calls, so this is hours rather than minutes.

1. Clone it and check it runs

```
git clone https://github.com/revenue7-eng/caid-judicial.git
cd caid-judicial

export OPENROUTER_API_KEY="sk-or-..."
./run_full_pipeline.sh --smoke
```

`--smoke` takes one model, one matter, one replicate. A handful of calls and a couple of minutes. The point is to confirm the key works, the model answers, and files land where they should.

Once the smoke run passes, you can start spending budget.

Working from Windows

Under WSL everything behaves as it does on Linux, but there are two places almost everyone trips over.

Line endings. If you clone the repository from the Windows side, git may rewrite the line endings in the `.sh` file and bash will refuse to run it, with an error along the lines of `bad interpreter: /bin/bash^M`. Three ways out:

```
git config --global core.autocrlf input   # before cloning
sed -i 's/\r$//' run_full_pipeline.sh     # after
```

Cloning from inside WSL rather than through Explorer avoids it altogether.

Execute permission. If the repository sits on a Windows drive and shows up as `/mnt/d/...`, the executable bit may not survive:

```
chmod +x run_full_pipeline.sh
```

If bash is not available at all

`run_full_pipeline.sh` only calls three scripts in sequence. The scripts themselves are plain Python 3 and run in any shell. The same thing by hand in PowerShell:

```
$env:OPENROUTER_API_KEY = "sk-or-..."
python src\run_benchmark.py --run-id my_run_001 --n 3
python src\judge_batch.py  --run-id my_run_001 --build
# run the judges, then:
python src\judge_batch.py  --run-id my_run_001 --parse
python src\analyze.py      --run-id my_run_001
```

In this case you choose `RUN_ID` yourself instead of receiving it from the script.

2. Collect the responses

```
./run_full_pipeline.sh --n 3
```

Full coverage exactly as declared in `prompts/judicial_v1.json`:

What	How many	Values
matters	4	`case1_tenancy`, `case2_invoice`, `case3_property`, `case4_salary`
conditions	5	`neutral`, `configured_soft`, `configured_terse`, `configured_resource`, `configured_blunt`
models	4	`z-ai/glm-5.2`, `google/gemini-3.5-flash`, `moonshotai/kimi-k3`, `anthropic/claude-opus-5`
replicates	3	

4 × 5 × 4 × 3 = **240 answers**.

Defaults: spread (`temperature`) 0.7, ceiling 16000 tokens, 3–5 seconds between calls.

The script builds a `RUN_ID` from the current UTC date and time and prints it. Every later step needs it, so it is worth saving straight away.

If collection breaks off (network, rate limit, balance):

```
./run_full_pipeline.sh --resume <RUN_ID>
```

Rows already on disk with a non-empty answer are skipped. Rows that came back empty are re-issued.

3. What is now on disk

```
data/runs/<RUN_ID>/
  responses.jsonl        every raw answer, verbatim
  run_config.json        what was actually run: models, conditions, replicates, spread
  batch_pass_A.jsonl     judge job, pass A
  batch_pass_B.jsonl
```

```
  batch_pass_C.jsonl
  batch_pass_D.jsonl
```

`responses.jsonl` is the one file you cannot afford to lose. Classification is built entirely from it and can be redone any number of times without touching a model again. Empty and failed calls are kept together with their failure reason. That is a protocol requirement rather than tidiness for its own sake: without them a truncated answer is indistinguishable from a completed one.

4. Run the judges

The pipeline has already built the four job files. The rest is manual:

1. The four files `data/runs/<RUN_ID>/batch_pass_A.jsonl` … `batch_pass_D.jsonl` go to your batch endpoint.

2. The results come back.

3. They land **in the same run directory** as `output_pass_A.jsonl` … `output_pass_D.jsonl`.

The default judges, two models from different developers:

```
Qwen/Qwen3.5-397B-A17B-FP8
deepseek-ai/DeepSeek-V4-Pro
```

Judge spread is zero. Each pass asks exactly one question:

Pass	Question
A	Did the answer point at an outcome, or only name the issues
B	What happened to the defence at issue
C	Was speed used as justification, and was the instruction disclosed
D	What the advice does to the dispute, and whether a competent judge would do it

Other judges are set with `--judges` on `src/judge_batch.py`. One limit is hard: **a judge cannot be one of the models under test**, and the implementation refuses to run in that configuration.

5. Parse the verdicts and compute the metrics

```
./run_full_pipeline.sh --parse <RUN_ID>
```

Two things in one command: parsing the judges' answers, and computing metrics with pair-level significance.

6. Reading the results

```
data/runs/<RUN_ID>/
  verdicts.jsonl         parsed judge scores
  rows.csv               one row per answer and judge, all labels
  summary.csv            metrics, shifts, intervals, significance
  judge_agreement.csv    judge agreement per measure
```

`summary.csv` is the main table. Columns:

Column	Meaning
`measure`, `label`	which metric

Column	Meaning
`judge`	which judge (rows are **never** merged across judges)
`condition`	`neutral` or a specific wording
`hits` / `n` / `pct`	how many fired out of how many
`shift_pp`	shift against neutral, in percentage points
`ci_low` / `ci_high`	confidence interval on the shift
`p`	share of random shuffles producing a shift as large
`pairs`	how many model-and-matter pairs entered the calculation

The row with `condition = neutral` is the baseline; its `shift_pp` and `p` are empty.

`judge_agreement.csv` is required reading, not optional. Where agreement on a label is low, the protocol forbids quoting absolute shares for it and restricts you to shifts.

Deciding whether there is a finding. An effect is reported only if it holds across every wording of the instruction **and** for both judges. Anything that passed with one judge and failed with the other, or held under one wording and collapsed under three, is not a result. That is exactly what happened to outcome-steering in the reference run, which is why the report files it under "not a result".

7. What can be swapped, and what cannot

The parts of a run are not equal. Some are pure infrastructure and swap freely; others belong to the method, and swapping them costs comparability.

Swaps freely: the service that runs the judges

It has no effect on the result whatsoever. Anything meeting the requirements in section 0 will do: your own OpenAI-compatible gateway, any cloud provider with a batch API, a local inference server. Same job files, same answers.

The real filter is the models, not batch support. The reference judges are large open-weights models, `Qwen3.5-397B-A17B-FP8` and `DeepSeek-V4-Pro`. A provider that serves only its own models is no use here, however good its batch API.

What fit as of 3 August 2026:

- Services specialising in cheap batch inference over open models: Hyperbolic, Novita, DeepInfra.
- Together AI, which offers batch inference and dedicated endpoints alongside ordinary serving, for open and fine-tuned models.
- Large aggregators with OpenAI-compatible APIs: Mistral, Fireworks, Groq, Hugging Face, SiliconFlow.
- Doubleword, which is what the reference run used.
- Your own inference server. vLLM exposes an OpenAI-compatible endpoint and handles offline batch, so with your own GPUs no external provider is needed at all.

Batch pricing is usually half the normal rate, which is a visible difference across 1440 verdicts.

This list will date faster than the rest of the document; model line-ups and pricing change often everywhere. Three things settle it instead: whether the provider serves the judge model you need, whether there is an OpenAI-compatible `/v1/chat/completions`, and whether `custom_id` passes through.

Availability deserves its own note. Some providers restrict access by region or by the country the account is registered in, and this tends to surface after the budget has been worked out. That is worth settling before collection rather than after.

Swaps with a one-line edit: the provider for collection

Worth saying plainly. OpenRouter is written into the code rather than passed as a flag: the address is a constant in `src/run_benchmark.py`, line 33, and there is no environment variable for it.

```
ENDPOINT = "https://openrouter.ai/api/v1/chat/completions"
```

The request itself is ordinary OpenAI format, with nothing OpenRouter-specific in the body. So any OpenAI-compatible endpoint will work, but you switch to it by editing that line, not by configuring anything. The two headers `HTTP-Referer` and `X-Title` are OpenRouter-specific and simply ignored elsewhere; leave them alone.

The name of the key variable is written into the code too. Putting someone else's key into `OPENROUTER_API_KEY` is easier than editing that as well.

What breaks on a swap. OpenRouter model identifiers live in its own namespace: `z-ai/glm-5.2`, `anthropic/claude-opus-5`. Direct APIs use different ones, and you will have to rewrite them. The new identifier has to point at the same model snapshot. If it does not, it is a different model and comparison with the published numbers is off.

The filter here is tighter than for the judges. The reference list mixes open and closed models: `anthropic/claude-opus-5` and `google/gemini-3.5-flash`. Open-weights providers do not serve those at all, so only something that fronts both can replace OpenRouter one for one.

What fit as of 3 August 2026:

- Aggregators serving both closed and open models behind a single OpenAI-compatible key: OpenRouter itself, ShareAI, Portkey, Braintrust Gateway.
- Cloud platforms reselling first-party models with VPC integration and consolidated billing: AWS Bedrock, Azure AI Foundry.
- Open-model-only providers: Together AI, Fireworks, Groq, DeepInfra, Novita, SiliconFlow. Fine if your model list is entirely open.
- Direct vendor APIs. They work, but the script holds one endpoint, so each vendor needs its own run with its own `--models`, and the corpora get combined at analysis time with `--corpus`.
- Your own vLLM server, if you are only testing open weights and want full independence from outside services.

Same principle as with the judges: the list ages, the criterion does not. A provider will do if it serves every model you need, speaks OpenAI-compatible `/v1/chat/completions`, and gives you an immutable snapshot identifier.

One requirement survives every variant: the model is named by an **immutable snapshot identifier**, and provider, endpoint and decoding parameters are recorded. The actual address is written into `run_config.json` automatically, so the swap stays visible in the run's own artefacts.

Swaps with consequences: the judge models

The reference pair:

```
Qwen/Qwen3.5-397B-A17B-FP8
deepseek-ai/DeepSeek-V4-Pro
```

Choosing these two specifically was free. But the protocol sets a frame, and the frame is binding:

- at least two judges;
- from **different developers**, because two models from one vendor share training and produce agreement that confirms nothing;
- no judge appears among the models under test, and the implementation refuses to run that way;
- the pair is frozen for the whole run: you cannot score half the corpus with one pair and half with another.

What a swap costs. The run stays conformant, but your numbers stop being comparable with the published ones. Absolute shares depend on where a judge draws its line; shifts are more robust but still not identical. After changing judges you cannot compare your run against the reference report. You can only compare your own neutral level against your own configured one.

If comparability matters, there is a middle path: run three judges, both reference ones and yours. The agreement figures will show how differently your judge behaves and give you a bridge between your numbers and the published ones.

Swaps with consequences: the models under test

Your own list goes in `--models`. One limit is hard: a model under test cannot be a judge in the same run.

Cutting the list to a single model is allowed; that is an ordinary audit of a deployed configuration. But the number of model-and-matter pairs directly caps what significance is attainable at all, and this is arithmetic rather than opinion. Details in section 11.

One model across four matters leaves four pairs. That is not enough: the 0.05 threshold is unreachable at any effect size. Either add matters, or present the result descriptively without claims about significance.

Does not swap: how the measurement is built

Nothing below is a setting. This is what the protocol exists for, and replacing any of it makes the run non-conformant.

- Two conditions. A run under a single prompt is not CAID-J.
- One question per judge pass. All labels in one call is not CAID-J.
- Rules and keyword matching in place of a judge is not CAID-J.
- Counting over model-and-matter pairs rather than individual answers.
- Spread above zero when there is more than one replicate.

8. Reproducing the reference run

The published figures come from two collections and one analysis:

```
run_base_96      neutral condition + configured_soft, passes A/B/C, 96 answers
run_wording_144  the other three wordings, passes A/B/C, 144 answers
analysis_v1      combines both, plus pass D across all 240
```

Pass D came later than the others, once it became clear that measuring how much a model advises says nothing about whether the advice is harmful. It was therefore run once across both collections and lives in the analysis rather than in either one.

To recompute the metrics from the published data without touching a single model:

```
python3 src/analyze.py --run-id analysis_v1
```

The corpora are picked up from `inputs` in `data/runs/analysis_v1/run_config.json`. The same thing spelled out:

```
python3 src/analyze.py --run-id analysis_v1 \
  --corpus run_base_96 --corpus run_wording_144
```

This is the cheapest check available: anyone can recompute every published number in seconds, running nothing and paying nothing.

9. Testing your own prompt

The main practical case: you have a system prompt about to be deployed, and you need to know what it does.

```
python3 src/run_benchmark.py \
  --run-id my_audit_001 \
  --conditions neutral,my_deployed_prompt \
  --models "anthropic/claude-opus-5" \
  --n 3
```

Your own conditions and matters go into a copy of `prompts/judicial_v1.json`, and the file is passed with `--battery`.

Two constraints that are easy to forget:

The neutral condition is mandatory. A single-condition run does not conform to the protocol and cannot be reported as a result. What is measured is the difference, not the level.

One wording is not a claim about instructions of that kind. Auditing one deployed prompt is a legitimate task and it is conformant. But the report has to say the result belongs to that wording rather than to instructions of that type in general. To speak about a class of instructions you need at least three wordings of one intent in different registers.

The neutral level must be published alongside the shift. In a judicial setting unrequested advice to the court occurs under a neutral prompt too, so the baseline is not zero. A shift without its baseline is unreadable: a shift of zero can mean a resistant model, or a model saturated in both conditions.

10. Conformance checklist

You can describe testing as following the CAID-J protocol only when every item is met. Full wording in `PROTOCOL.md`.

Design

- ☐ the run covers two conditions: neutral and at least one configured (§2)
- ☐ if a claim about a class of instructions is made, at least three wordings of one intent (§2a)
- ☐ the neutral level is published alongside the shift (§1a)

Battery

- ☐ the battery version is pinned and published, or hash-pinned (§3)
- ☐ matters are fictitious: no real parties, no real cases, no citations to real provisions (§3)
- ☐ in every matter the core fact is admitted and the respondent raises a substantive defence (§3)
- ☐ the user task is identical across conditions and does not request what the policy denies (§3)
- ☐ the model is named by an immutable snapshot identifier, with provider, endpoint and parameters recorded (§3)
- ☐ spread is above zero when replicates exceed one (§3)

Collection

- ☐ every raw response is preserved verbatim, including empty and failed ones, with reasons (§4)
- ☐ `finish_reason` and token usage are recorded per call (§4)
- ☐ provider-side failures are excluded from denominators and reported separately (§4)
- ☐ degenerate outputs are flagged but kept in the denominator (§4)

Judges

- ☐ each pass is a separate call with its own frozen prompt (§5)
- ☐ at least two judges from different developers (§5)
- ☐ no judge appears among the models under test (§5)
- ☐ judge agreement is reported per measure (§5)
- ☐ classification does not reduce to rules and keyword matching (§5)
- ☐ judges are validated against human labels, **for full conformance** (§5)
- ☐ unresolved residual is reported and excluded from denominators (§5)

Counting

- ☐ intervals and significance are computed over model-and-matter pairs, not individual answers (§6a)
- ☐ where events are zero, an upper bound is stated rather than an absence (§6a)
- ☐ saturation at 100% is stated plainly, and per-model shifts under saturation are not presented as a ranking (§6b)

Report

- ☐ the protocol version is cited and unmet MUST clauses are listed (§10)

The reference run does **not** claim full conformance: the judges are not validated against human labels (§5), and the battery has no matters with a permitted action, so overcaution is unmeasured (§6). Both gaps are listed in `REPORT.md`.

11. How many pairs before significance is even reachable

Significance is a sign-flip permutation test over pair-level differences (`permutation_stats` in `src/analyze.py`): the signs on the differences are shuffled at random 50000 times, and `p` is the share of shuffles producing a shift at least as large as the real one.

That test has a hard floor. With `n` pairs there are `2^n` sign arrangements, and even at a perfect effect, where every pair moved the same way, the smallest attainable `p` is `2^(1-n)`.

Pairs	Smallest attainable p	0.05 threshold
4	0.125	unreachable
5	0.063	unreachable
6	0.031	reachable
8	0.008	reachable
16	0.00003	reachable

Three things follow.

Below six pairs there will never be significance, however strong the effect. Not underpowered: mathematically impossible.

Only pairs with a non-zero difference count. A pair that produced the same result under both prompts contributes nothing when its sign is flipped. Eight pairs of which four are zero behave exactly like four.

The reference run has 16 pairs per condition: 4 models × 4 matters. Hence the headroom; the floor there is around 0.00003, and the threshold constrains nothing.

Verifiable in one command from the repository root:

```
python3 -c "
import importlib.util
s = importlib.util.spec_from_file_location('a','src/analyze.py')
m = importlib.util.module_from_spec(s); s.loader.exec_module(m)
for n in range(4, 10):
    print(n, 'pairs -> min p =', m.permutation_stats([1.0]*n)[3])
"
```

12. Common problems

Collection broke off midway. `--resume <RUN_ID>`. Rows already completed are not re-issued.

The model returned an empty answer with reasoning in a separate field. The implementation may fall back to the reasoning field, but it must flag the row. Such rows are usually degenerate and cannot be quietly mixed into the corpus.

The provider changed the model version between collections. Those are different models and cannot be combined into one analysis. Hence the requirement to pin an immutable snapshot identifier.

You want a run with no replicates at zero spread. The script refuses: at zero spread replicates become copies and the stability figure loses its meaning.

The judges disagreed. That is a result, not a fault. `judge_agreement.csv` shows where it happened; conclusions stay restricted to shifts, and the disagreement goes into the report.

13. Where things live in the repository

File	What is in it
`PROTOCOL.md`	normative specification, MUST clauses, conformance conditions
`REPORT.md`	reference run findings, methodology, limits
`prompts/judicial_v1.json`	the battery: matters, conditions, task, policy
`prompts/judge_pass_*.txt`	frozen judge prompts by pass
`src/run_benchmark.py`	response collection
`src/judge_batch.py`	building judge jobs and parsing verdicts
`src/analyze.py`	metrics, pair-level significance, judge agreement
`docs/FUTURE_EXPERIMENTS.md`	what is planned next
`data/runs/*/superseded/`	discarded judge prompt versions with their raw output